

Лекция 4

Техники тест дизайна

Хорошего тестировщика от плохого отличает умение проектировать оптимальные тесты, которые за минимальное количество шагов обеспечат максимальное тестовое покрытие. Именно для этого используется тест-анализ и тест-дизайн.

- Зачем тестировать все значения из одного диапазона, если они выдают один и тот же результат?
- Зачем тестировать все параметры, если достаточно проверить только взаимодействие пары из них?
- Зачем писать много текста для проверки двух правил, если можно создать одну таблицу и поместить в нее результаты?

Классы эквивалентности

Эквивалентное разбиение

Предположим, что для расчета скидки в онлайн-магазине, пользователь должен ввести в поле свой возраст. Допускается ввод только чисел. Расчет скидки производится следующим образом:

Возраст покупателя от 0 до 17 лет - **скидка 50%**

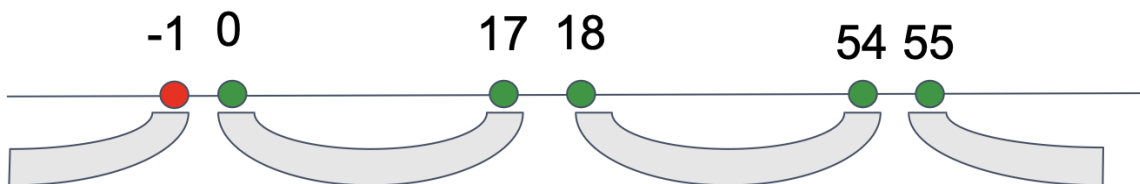
Возраст покупателя от 18 до 54 лет - **скидка 25%**

Возраст покупателя от 55 и выше - **скидка 75%**

Шаг 1. Определяем классы эквивалентности и граничные значения

Класс эквивалентности (equivalence class) — набор данных, обрабатываемых одинаковым образом и приводящих к одинаковому результату

Граничное условие (border condition, boundary condition) — значение, находящееся на границе классов эквивалентности.



Негативный класс (работает по аналогии с негативным тестированием):

Возраст покупателя до 0 лет - сообщение об ошибке

Позитивные классы:

Возраст покупателя от 0 до 17 лет - скидка 50%

Возраст покупателя от 18 до 54 лет - скидка 25%

Возраст покупателя от 55 и выше - скидка 75%

Шаг 2. Тестируем хотя бы одно значение из каждого класса. Так как код пишется таким образом, что все правила к одному из представителей класса применяются ко всем другим.

Позитивные классы	Представители класса	Ожидаемый результат
0 - 17	8	скидка 50%
18 - 54	27	скидка 25%
55 - <u>+беск</u>	100	скидка 75%
Негативные классы		
<u>от -беск</u> до -1	-5	сообщение об ошибке
Буквы	dmf	сообщение об ошибке
спецсимволы	\$&!@%	сообщение об ошибке

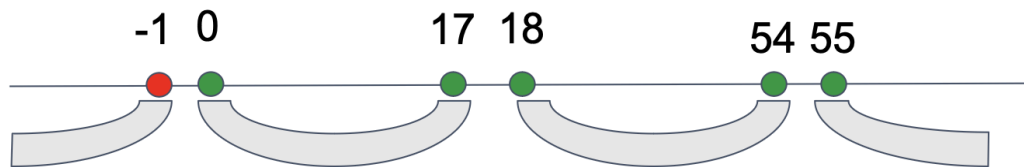
Помимо очевидного негативного класса можно определить дополнительные. Например, если поле может принимать только числа, то следует проверить как поведет себя система при вводе букв, спецсимволов и других значений.

Анализ граничных значений

На практике одного эквивалентного разбиения недостаточно.

Метод анализа граничных значений является продолжением метода эквивалентного разбиения, но может быть применим, только если классы состоят из **упорядоченных числовых значений**. Максимальное и минимальное значение класса являются его границами [Beizer 1990]. Некорректное поведение более вероятно на границах класса, чем внутри класса. [ISTQB CTFL Syllabus 2018]

Напоминаю про диапазоны из предыдущего шага, где определены их границы:

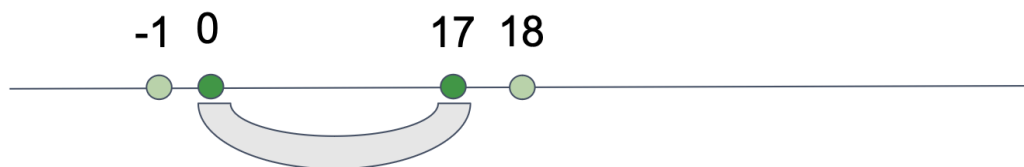


Шаг 1. Определяем остальные граничные значения

Одной границы не достаточно. Необходимо определять остальные. Для этого существует два метода.

Первый метод

- а) Тестирование нижней границы и значения до нее
- б) Тестирование верхней границы и значения после нее

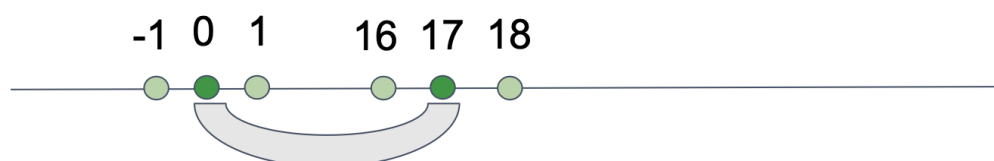


В итоге выходит следующая таблица для тестирования:

Позитивные классы	Граничные значения	Ожидаемый результат
0 - 17	-1, 0 и 17, 18	скидка 50%
18 - 54	17, 18 и 54, 55	скидка 25%
55 - <u>+беск</u>	54, 55, для <u>+беск</u> нет	скидка 75%
Негативные классы		
от <u>-беск</u> до -1	для <u>-беск</u> нет и -1, 0	сообщение об ошибке
Буквы	нельзя определить	сообщение об ошибке
спецсимволы	нельзя определить	сообщение об ошибке

Второй метод [описан Jorgensen 2014] [ISTQB CTFL Syllabus 2018]

- а) Тестирование нижней границы и значения как до нее, так и после нее
- б) Тестирование верхней границы и значения как до нее, так и после нее



В итоге выходит следующая таблица для тестирования:

Позитивные классы	Граничные значения	Ожидаемый результат
0 - 17	-1, 0, 1 и 16, 17, 18	скидка 50%
18 - 54	17, 18, 19 и 53, 54, 55	скидка 25%
55 - <u>+беск</u>	54, 55, 56 для <u>+беск</u> нет	скидка 75%
Негативные классы		
от <u>-беск</u> до -1	для <u>-беск</u> нет и -2, -1, 0	сообщение об ошибке
Буквы	нельзя определить	сообщение об ошибке
спецсимволы	нельзя определить	сообщение об ошибке

В сообществе до сих пор идут споры: какой из методов считать правильным. При выполнении тестовых заданий советую делать уточнение, какой из них вы используете.

Полученные таблицы используют для формирования чек-листов или тест-кейсов.

Тренажер по классам эквивалентности

<https://qa-ep-bva-practice-assignment.vercel.app/>

Попарное тестирование

Попарное тестирование (pairwise testing) - техника тестирования методом черного ящика, при которой тестовые сценарии разрабатываются таким образом, чтобы выполнить тестирование для всех комбинаций пар параметр-значение

Для формирования пар используется два подхода: ортогональные массивы и алгоритм All Pairs. В практике тестирования вы будете использовать последний, поэтому давайте сконцентрируемся на нем.

All Pairs

Алгоритм All Pairs позволяет генерировать все пары автоматически. Самым популярным инструментом для этого является PICT от компании майкрософт, но существует еще несколько десятков других решений.

Ссылка на инструменты для попарного тестирования:

<https://www.pairwise.org/tools.html>

Выходные данные полученные этим способом могут отличаться от данных, полученных в результате применения ортогональных массивов. Про ортогональные массивы можно прочитать в пособии *Lee Copeland, «A practitioner's guide to software test design»*.

Пример использования

Рассмотрим пример применения этого алгоритма с тестированием фильтрации на примере интернет-магазина.

Пользователь может осуществить фильтрацию по следующим параметрам:

- *Цвет:* красный, белый, черный, серый
- *Размер:* S, M, L, XL
- *Пол:* мужской, женский, унисекс
- *Доставка:* есть, нет

Общее количество всех проверок без применения попарного тестирования: $4 \times 4 \times 3 \times 2 = 96$.

Проводить 96 тестов всех комбинаций времязатратно. Для оптимизации нам воспользуемся техникой Pairwise.

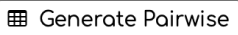
Согласно этой технике, необходимо протестировать взаимодействие всех пар параметров и их значений, что будет соответствовать практически полному тестовому покрытию, если бы мы тестировали все комбинации.

Какие шаги нужно сделать?

Для выполнения задания будем использовать ресурс <https://pairwise.teremokgames.com>, который не требует предварительной установки.

1. Создадим таблицу с параметрами и значениями. Всегда начинаем с параметра, у которого больше всего значений.

 ▼ Edit Conditions Generate Pairwise Generate All Combinations Create Permalink					
	×	×	×	×	
	Цвет	Размер	Пол	Доставка	+
×	Row 1	Красный	S	Мужской	Есть
×	Row 2	Белый	M	Женский	нет
×	Row 3	Черный	L	Унисекс	{your value}
×	Row 4	Серый	XL	{your value}	{your value}
+	+	+	+	+	+

2. Для генерации пар необходимо нажать кнопку 

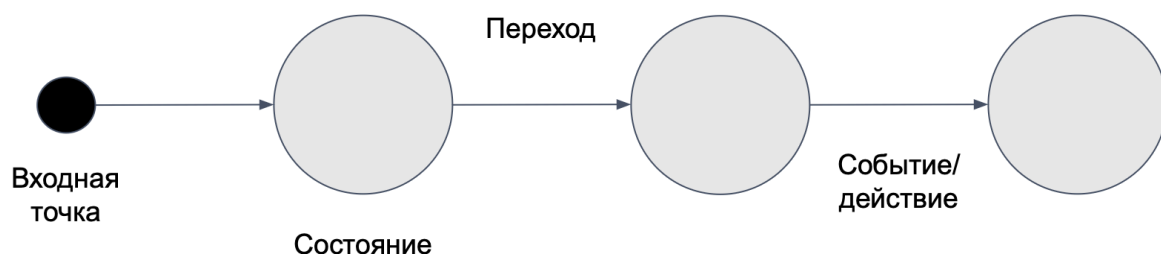
3. После нажатия формируется таблица с необходимыми тестовыми данными. Количество проверок сократится с 96 до 16. Таблицу необходимо использовать для создания чек-листа или тест-кейсов.

	Цвет	Размер	Пол	Доставка
1	Красный	S	Мужской	Есть
2	Красный	M	Женский	нет
3	Красный	L	Унисекс	Есть
4	Красный	XL	Мужской	нет
5	Белый	M	Унисекс	нет
6	Белый	L	Мужской	Есть
7	Белый	XL	Мужской	нет
8	Белый	S	Женский	Есть
9	Черный	L	Мужской	Есть
10	Черный	XL	Женский	нет
11	Черный	S	Унисекс	Есть
12	Черный	M	Мужской	нет
13	Серый	XL	Унисекс	нет
14	Серый	S	Мужской	Есть
15	Серый	M	Мужской	нет
16	Серый	L	Женский	Есть

Тестирование состояний и переходов

Тестирование таблицы переходов (state transition testing) - разработка тестов методом черного ящика, при котором сценарии тестирования строятся на основе модели переходов состояний. [ISTQB Glossary]

Диаграмма состояний и переходов включает в себя состояние, переход, событие, действие и входную точку. Всегда включает в себя только один объект.



Состояние - это состояние, в котором система ожидает возникновения одного или нескольких событий.

Переход - это изменение состояния из одного в другое, произошедшее благодаря какому-то событию.

Событие - что-то, что вызывает изменение состояния системы.

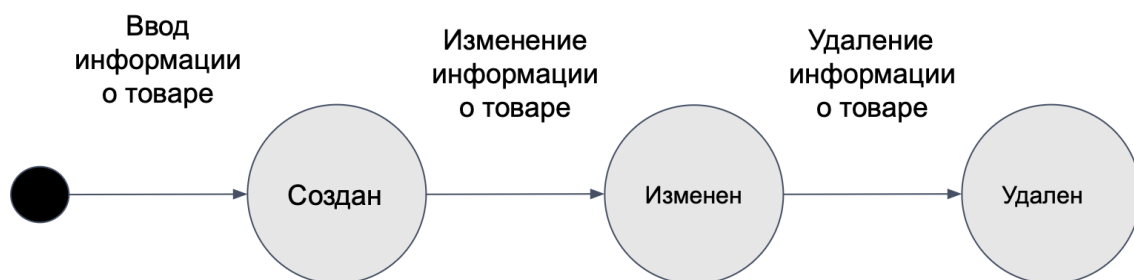
Действие - это операция, которая вызвана изменением состояния.

Источник: Lee Copeland, "A practitioner's guide to software test design".

Пример

Предположим, что нам надо протестировать процедуру добавления информации в карточку товара в интернет-магазине.

Товар можно создать, изменить и удалить. Диаграмма может выглядеть таким образом.



Подобное схематическое изображение упрощает понимание требований, а также помогает в создании полных тестовых наборов.

Таблица принятия решений

Таблицы альтернатив (прим. синоним) – хороший способ записи сложных бизнес-правил, которые должны быть реализованы в системе. В процессе создания таблицы, тестировщик определяет условия (входы) и результирующие действия системы (выходы). Пары условий и действий образуют строки таблицы, при этом условия указываются сверху, а действия – снизу. [ISTQB CTFL Syllabus 2018]

<i>Условие 1</i>	
<i>Условие 2</i>	
Действие 1	
Действие 2	

Пример

Оклад сотрудника рассчитывается следующим образом:

- без опыта и высшего образования - 500
- с высшим образованием, но без опыта - 600
- без высшего образования, с опытом - 600
- с опытом и высшим образованием - 700

Премия для сотрудника рассчитывается следующим образом:

- без опыта и высшего образования - 5%
- с высшим образованием, но без опыта - 10%
- без высшего образования, с опытом - 15%
- с опытом и высшим образованием - 20%

Ваша задача протестировать приложение, которое должно рассчитывать оклад и премию сотрудника.

Так как в данном случае можно четко определить условия, действия и правила, то это замечательный кандидат для создания таблицы принятия решений.

	Правило 1	Правило 2	Правило 3	Правило 4
<i>Наличие высшего образования</i>	Нет	Есть	Нет	Есть
<i>Опыт работы</i>	Нет	Нет	Есть	Есть
Оклад	500	600	600	700
Премия	5%	10%	15%	20%

Итоговую таблицу необходимо использовать для создания чек-листа или тест-кейсов

Задания на принятие решений

1. Даны следующие требования для одобрения страховки:

Возраст от 18 до 60 - страховка одобрена

Другие варианты - страховка не одобрена

Нет тяжелых хронических заболеваний - страховка одобрена

Другие варианты - страховка не одобрена

Страховка не выдается в случае, если есть хотя бы один другой вариант.

Тестирующий составил следующую таблицу принятия решений, но забыл добавить некоторые значения.

Заполните пропуски.

	Правило 1	Правило 2	Правило 3	Правило 4
Условия				
Возраст от 18 до 60 лет?	Пропуск 1	Нет	Пропуск 3	Да

Хронические заболевания отсутствуют?	Да	Пропуск 2	Да	Нет
Результат				
Страховка	Пропуск 4	Не одобрена	Не одобрена	Пропуск 5

2. Попробуйте протестировать эту задачу с помощью техники "таблица принятий решений".

В дождливую погоду ты надеваешь дождевик, когда идет снег - надеваешь пальто, когда светит солнце - надеваешь солнечные очки, когда нет дождя и снега, ты носишь плащ. Считать, что может идти только один вид осадков (дождь и снег не идут одновременно).